Lab Report: Week 7 Session A
Capstone Project Planning

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 7 Session A: **capstone planning** for the Smart Water Integrated Monitoring and Decision Support Dashboard. Exercises completed: project scope (`SCOPE.md`), system architecture (`ARCHITECTURE.md`), development scaffold (`app/`, `src/`, tests, sample data), and repository setup instructions. The capstone reuses Weeks 3–6 modules (SCS-CN runoff, reservoir optimization, DEM flood inundation, rainfall alerts) behind a future Streamlit UI. AI collaboration used **Cursor Agent**; OpenCode was not required. Evidence: public GitHub repository (Figures ??–??), planning files in `ai_water_lab/capstone/`, and `prompt_log.md` (Appendix ??). Push to `main` completed successfully (30 objects).

1 Introduction

Week 7 Session A is a **planning lab**, not a full implementation sprint. The deliverable is a clear scope, architecture, scaffold, and GitHub repository so Session B can implement core features with documented AI prompts.

2 Objectives

- Define project scope and requirements (Exercise 1).
- Design system architecture with AI assistance (Exercise 2).
- Generate technical scaffold (Exercise 3).
- Initialize GitHub repository (Exercise 4).

3 Exercise 1: Project scope

3.1 Problem and users

Problem: Separate lab scripts for rainfall, runoff, reservoir dispatch, and flood mapping should be unified for teaching and demo purposes.

Users: Students, municipal flood-duty trainees, researchers prototyping integrated workflows.

3.2 Requirements summary

Non-functional: Python 3.8+, offline sample data, documented units, `prompt_log.md`, modular `src/`.

Success criteria: End-to-end demo on samples; pytest for hydrologic validity; public GitHub repo with README and `AGENTS.md`.

Out of scope: Production Vercel deploy, licensed GIS DEM, multi-reservoir networks.

Table 1: Functional requirements (from `SCOPE.md`).

ID	Requirement
F1	Weather / alerts from CSV or optional API
F2	SCS-CN runoff with physical checks
F3	7-day reservoir dispatch summary
F4	DEM inundation vs. `flood_level`
F5	Streamlit multi-tab UI

4 Exercise 2: Architecture

Logical flow: weather → runoff → reservoir → flood → Streamlit shell. Technology: Streamlit, NumPy, Pandas, SciPy, Matplotlib, pytest, GitHub. Internal Python APIs (`load_rainfall_csv`, `scs_runoff_mm`, `simulate_flood`) rather than REST for the teaching build.

Table 2: Module map to prior labs.

Capstone module	Prior lab folder
`src/weather`	`week5_session_a_lab1/`
`src/runoff`	`week5_session_b_lab2/`, `week3_session_b/`
`src/reservoir`	`week6_session_a_lab3/`
`src/flood`	`week6_session_b_lab4/`

Full diagram and data-flow detail: `ARCHITECTURE.md` (Appendix **??**).

5 Exercise 3: Technical scaffold

Created under `ai_water_lab/capstone/`:

- `app/main.py` — Streamlit stub with four tabs
- `src/weather`, `src/runoff`, `src/reservoir`, `src/flood`
- `tests/test_runoff.py`, `tests/test_flood.py`
- `data/sample_rainfall.csv`
- `requirements.txt`, `README.md`, `AGENTS.md`

Week 7 Session B will wire working logic and run `streamlit run app/main.py`.

6 Exercise 4: Repository

Planned repository name: `smart-water-capstone`.

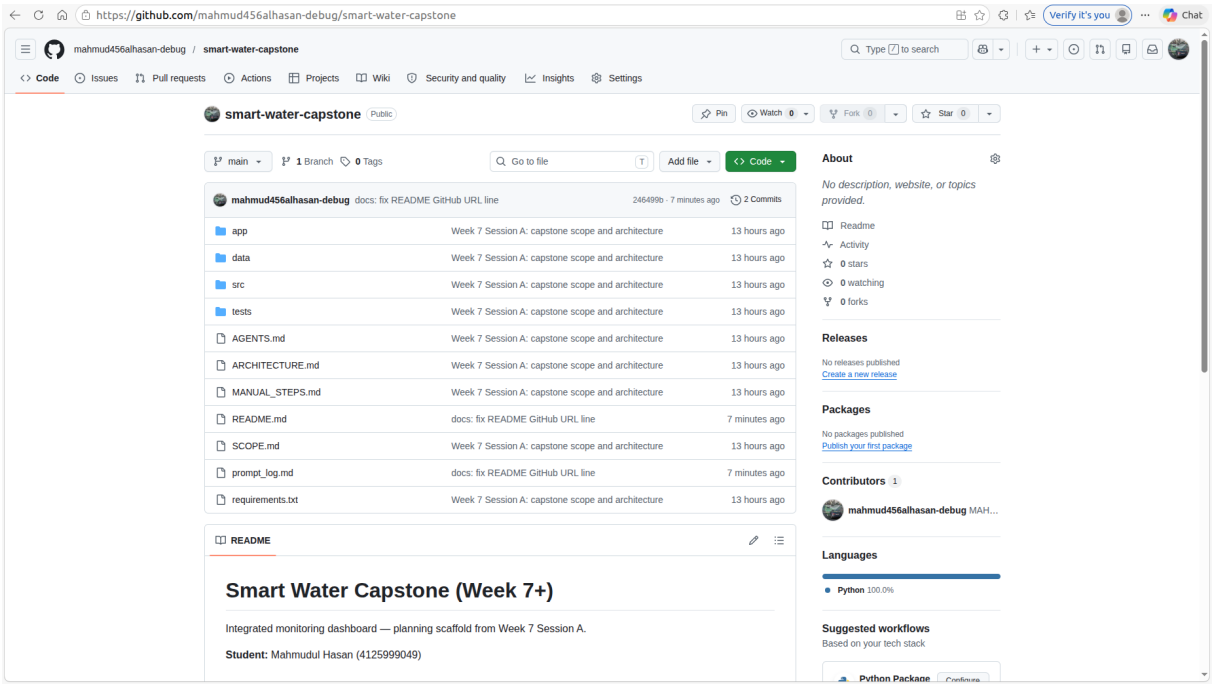
Repository URL: `https://github.com/mahmud456alhasan-debug/smart-water-capstone`

Branch: `main` (tracking `origin/main`)

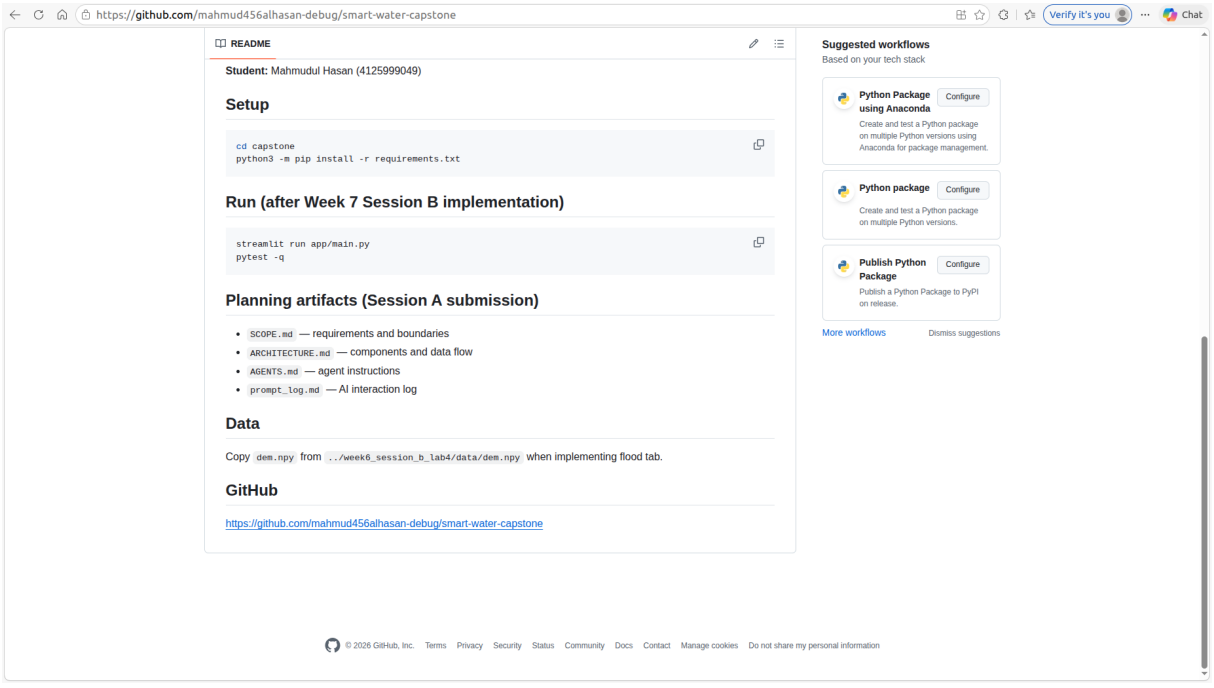
Initial commit: `Week 7 Session A: capstone scope and architecture` — 30 files/objects pushed via HTTPS with personal access token.

6.1 GitHub repository evidence

Figures **??** and **??** show the public repository file tree (planning markdown, `app/`, `src/`, `tests/`, `data/`) and the rendered `README.md` setup instructions.



(a) Repository root: `smart-water-capstone`, commit on `main`, Python project layout.



(b) Rendered `README.md`: setup, Session B run commands, planning artifacts list.

Figure 1: GitHub evidence for Exercise 4 (repository setup and push).

6.2 Prompt log

All AI prompts for Exercises 2–3 are in `prompt_log.md` (Appendix **??**). Tool: Cursor Agent; no OpenCode session required.

7 Discussion

1. Reusing four lab modules reduces capstone risk compared with a greenfield stack.
2. Session A intentionally leaves Streamlit tabs as stubs so planning time stays within one hour.
3. `AGENTS.md` prepares Session B agents for consistent units and file layout.
4. Two GitHub page screenshots (file tree + README) satisfy the lab guide's optional screenshot requirement.

8 Lessons learned

Define scope boundaries before architecture prompts. Keep CSV/sample-data defaults for reproducible grading. Document prompts in `prompt_log.md` as you go, not after Week 8. GitHub push and one screenshot remain manual even when AI generates scaffold and report text.

9 Conclusion

Week 7 Session A deliverables are complete: planning files in `ai_water_lab/capstone/`, public GitHub repository `https://github.com/mahmud456alhasan-debug/smart-water-capstone`, Figures **??**, and prompt log appendix. Submit this report PDF and repository link to the course platform. Implementation continues in Week 7 Session B.

A Scope document: `SCOPE.md`

SCOPE.md (full file)

```
# Capstone Scope -- Smart Water Integrated Dashboard

**Student:** Mahmudul Hasan (4125999049)
**Course:** AI-Augmented Software Engineering (Week 7 Session A)

## Project title

Smart Water Integrated Monitoring and Decision Support Dashboard

## Problem statement

Urban and watershed managers need one place to connect rainfall observations, estimated
↪  runoff, reservoir release trade-offs, and flood extent under a chosen water level. In
↪  Weeks 3--6 this work lived in separate Python scripts. The capstone unifies those
↪  modules in a single Streamlit application with reproducible sample data, physical
↪  sanity checks, and documented AI-assisted development (`AGENTS.md`, `prompt_log.md`).

## Target users

- Hydrology / water-resources students practicing AI-assisted development
- Municipal flood-duty staff reviewing thresholds and maps (demo / training, not
↪  operations)
- Researchers prototyping integrated screening workflows before GIS production tools

## Functional requirements
```

```
1. **Weather / alerts:** Load rainfall from CSV or optional API; show threshold-based
↪  alert status (from Week 5 Lab 1 pattern).
2. **Runoff:** Compute SCS-CN runoff for sample storms; enforce runoff <= rainfall (Week 3
↪  / Week 5 Lab 2).
3. **Reservoir:** Run 7-day dispatch optimization summary (Week 6 Lab 3; MCM storage,
↪  m3/s flows).
4. **Flood map:** Inundation from synthetic DEM + `flood_level` (Week 6 Lab 4).
5. **Navigation:** Streamlit multi-tab UI linking the four modules with consistent units
↪  in labels.

## Non-functional requirements

- Python 3.8+; install via `requirements.txt`
- Runs offline on bundled sample data (fixed seeds)
- Documented units (mm, m3/s, MCM, m) in UI and README
- `prompt_log.md` maintained for all major AI prompts (Week 7--8)
- Modular `src/` packages reusable from prior `ai_water_lab` labs

## Success criteria

- End-to-end demo on sample data without editing source before run
- At least one automated test for hydrologic validity (e.g. runoff <= rainfall)
- Public GitHub repository with README, `AGENTS.md`, and planning artifacts from Session A
- Live Streamlit demo acceptable on local host for Week 8 presentation

## Scope boundaries (out of scope)

- Production deployment on Vercel / Streamlit Cloud (optional stretch)
- Real-time national GIS or licensed DEM tiles (synthetic DEM only for capstone)
- Multi-reservoir network or stochastic optimization
- SMS / email alert delivery in production

## Prior lab reuse

| Module | Source folder |
|--------|----------------|
| Weather / alerts | `week5_session_a_lab1/` |
| SCS-CN runoff | `week5_session_b_lab2/`, `week3_session_b/` |
| Reservoir dispatch | `week6_session_a_lab3/` |
| Flood inundation | `week6_session_b_lab4/` |
```

B Architecture: ARCHITECTURE.md

ARCHITECTURE.md (full file)

```
# System Architecture -- Smart Water Capstone

## 1. Component diagram (logical)

```text
+------------------+     +------------------+
|  Weather / CSV   |---->|  Alert engine    |
+------------------+     +--------+---------+
                                  |
+------------------+     +--------v---------+
|  Rainfall series |---->|  SCS-CN runoff   |
+------------------+     +--------+---------+
                                  |
                         +--------v---------+
                         | Reservoir optim. |
                         +--------+---------+
```
```

```
                                  |
+------------------+     +--------v---------+
|  DEM (npy)       |---->| Flood inundation |
+------------------+     +--------+---------+
                                  |
                         +--------v---------+
                         | Streamlit UI     |
                         | (tabs / charts)  |
                         +------------------+
```

2. Directory structure

```text
capstone/
  README.md
  AGENTS.md
  requirements.txt
  prompt_log.md
  SCOPE.md
  ARCHITECTURE.md
  app/
    main.py                 # Streamlit entry, tab layout
  src/
    weather/                # load data, alert thresholds
    runoff/                 # SCS-CN, validation
    reservoir/              # horizon optimize, summary tables
    flood/                  # simulate_flood, plots
  data/
    sample_rainfall.csv
    dem.npy                 # copy or symlink from week6 lab
  tests/
    test_runoff.py
    test_flood.py
```

3. Module responsibilities

Module	Responsibility
`app/main.py`	Streamlit shell; sidebar settings; tab routing
`src/weather`	Parse CSV/API rainfall; compute alert level
`src/runoff`	SCS-CN Q(P, CN); clip Q when P < Ia
`src/reservoir`	Wrap `optimize_horizon`; display revenue / storage table
`src/flood`	`simulate_flood`, stats, matplotlib figures for UI
`tests/`	Physical checks and monotonic flood %

4. Data flow

1. User selects storm CSV or default sample → rainfall depth mm.
2. Runoff module returns Q mm; UI shows hydrograph or single-event Q.
3. Optional: pass inflow series into reservoir module → 7-day schedule table.
4. User sets `flood_level` (m) flood module reads `data/dem.npy` mask, depth, %
↪ flooded.
5. Alert tab reads latest rainfall intensity vs configured thresholds.

5. Technology stack

Layer	Choice
Language	Python 3.8+
UI	Streamlit
Numerics	NumPy, Pandas

```
| Optimization | SciPy (`minimize_scalar`) |
| Plotting | Matplotlib (embedded in Streamlit) |
| Tests | pytest |
| VCS | GitHub |
| AI workflow | Cursor / any LLM + `AGENTS.md` + `prompt_log.md` |

## 6. API design (internal Python, not REST)

| Function | Input | Output |
|----------|-------|--------|
| `load_rainfall_csv(path)` | file path | `DataFrame` datetime, mm |
| `compute_runoff(p_mm, cn)` | scalars/series | runoff mm |
| `run_reservoir_baseline()` | eco flow optional | schedule table, total revenue |
| `simulate_flood(dem, level)` | 2D array, float m | mask, depth |
| `evaluate_alerts(intensity_mm_h)` | float | GREEN / AMBER / RED |

External REST (OpenWeather) optional behind config flag; capstone defaults to CSV for
↪ reproducibility.
```

C Agent instructions: AGENTS.md

AGENTS.md (full file)

```
# AGENTS.md -- Smart Water Capstone

## Project overview

Integrated Streamlit dashboard combining rainfall/alerts, SCS-CN runoff, reservoir
↪ dispatch (SciPy), and DEM-based flood inundation. Built from `ai_water_lab` Weeks
↪ 3--6. Student: Mahmudul Hasan (4125999049).

## Tech stack

| Layer | Technology |
|-------|------------|
| UI | Streamlit |
| Data | pandas, numpy |
| Optimization | scipy |
| Plots | matplotlib |
| Tests | pytest |
| Python | 3.8+ |

## Directory structure

```text
capstone/
  app/main.py
  src/weather/  src/runoff/  src/reservoir/  src/flood/
  data/  tests/
```

Conventions

- Units: rainfall mm; reservoir storage MCM; flows m3/s; flood stage m (absolute, same as
↪ DEM).
- Reuse logic from week5/week6 labs; refactor into `src/` rather than copy-paste
↪ duplicates.
- Wet cell rule: `elevation <= flood_level`; depth zero off wet cells.
- Runoff must satisfy `Q <= P` where applicable.
- Commit messages: `Week N: short description`.
- Document every major AI prompt in `prompt_log.md`.
```

````
## Run commands

```bash
cd capstone
pip install -r requirements.txt
streamlit run app/main.py
pytest -q
```

## Week 7 Session B priorities

1. Wire `src/runoff` and `src/flood` with tests.
2. Streamlit tabs with sample data.
3. Integrate reservoir summary tab.
4. Optional weather API behind env flag.
````

D Streamlit stub: app/main.py

app/main.py (full file)

```
"""Streamlit entry point -- Week 7 Session B will implement tabs."""

import streamlit as st

st.set_page_config(page_title="Smart Water Capstone", layout="wide")
st.title("Smart Water Integrated Dashboard")
st.caption("Mahmudul Hasan (4125999049) -- capstone scaffold (Week 7 Session A)")

tab1, tab2, tab3, tab4 = st.tabs(
    ["Weather & Alerts", "Runoff (SCS-CN)", "Reservoir", "Flood map"]
)

with tab1:
    st.info("Week 7 Session B: wire src.weather")
with tab2:
    st.info("Week 7 Session B: wire src.runoff")
with tab3:
    st.info("Week 7 Session B: wire src.reservoir")
with tab4:
    st.info("Week 7 Session B: wire src.flood")
```

E Submitted prompt log: prompt_log.md

prompt_log.md (full file)

````
# Prompt Log - Week 7 Session A (Capstone Planning)

**Student:** Mahmudul Hasan (4125999049)
**Tool:** Cursor Agent (Auto); OpenCode not used

---

## Exercise 1 - Project scope

**AI used?** Partial  scope drafted from completed Weeks 3--6 lab work, then saved to
↪  `SCOPE.md`.
````

````
**Prompt (summary):** Integrate week5_session_a_lab1 (weather/alerts),
↪  week5_session_b_lab2 (SCS-CN), week6_session_a_lab3 (reservoir), week6_session_b_lab4
↪  (flood) into one Streamlit capstone; list functional/non-functional requirements and
↪  scope boundaries.

**Output file:** `SCOPE.md`

**Changes made:** Fixed title "Smart Water Integrated Dashboard"; excluded production
↪  deployment and real GIS DEM from scope.

---

## Exercise 2 - Architecture design

**Date:** 2026-05-21

**Prompt (from lab guide, filled):**

```text
I want to build a Smart Water Integrated Monitoring and Decision Support Dashboard.

Requirements:
1. Load or fetch rainfall/weather data (CSV default; optional API).
2. Compute SCS-CN runoff with physical checks (runoff <= rainfall).
3. Run 7-day reservoir release optimization (scipy; MCM storage; m3/s flows).
4. Flood inundation from 100x100 synthetic DEM and flood_level (m).
5. Streamlit UI with tabs: Weather/Alerts, Runoff, Reservoir, Flood map.
6. prompt_log.md and AGENTS.md for AI-assisted development.

Please help me design:
1. System architecture diagram (describe components)
2. Directory structure
3. Main modules and their responsibilities
4. Data flow between components
5. Technology stack recommendations
6. API design (if applicable)
```

**Summary:** Five-layer flow (weather  runoff  reservoir  flood  Streamlit). Modular
↪  `src/` packages mapped to prior labs. Internal Python function API; CSV-first for
↪  reproducibility.

**Output file:** `ARCHITECTURE.md`

**Changes made:** ASCII diagram for LaTeX/report compatibility; no FastAPI (unlike week2
↪  demo) to keep capstone scope teachable.

---

## Exercise 3 - Technical plan / scaffold

**Prompt (from lab guide):**

```text
Based on our architecture design, please generate:
1. Complete directory structure
2. Main file templates with docstrings
3. requirements.txt with all dependencies
4. README.md with setup instructions
5. Initial AGENTS.md template
6. Sample data files (if needed)
````
````

```
Make it ready for development in Week 7 Session B. Python 3.8+, Streamlit, numpy, pandas,
↪  matplotlib, scipy.
```

Summary: Created `app/main.py` stub, `src/{weather,runoff,reservoir,flood}` packages,
↪ `tests/`, `data/sample_rainfall.csv`, README, AGENTS.md, requirements.txt.

Changes made: Stubs only | implementation deferred to Session B.

Exercise 4 - Repository

GitHub URL: https://github.com/mahmud456alhasan-debug/smart-water-capstone

Planned repo name: `smart-water-capstone`

Commit message: `Week 7 Session A: capstone scope and architecture`

Push result: `main` branch, 30 objects, tracking `origin/main`.

Report screenshots: `week7_session_a_github_page_a.png` (file tree),
↪ `week7_session_a_github_page_b.png` (README).

Lessons learned

Planning works best when capstone scope explicitly reuses lab modules instead of starting
↪ a new stack. Documenting wet-cell and unit conventions early avoids Week 8 debugging.
↪ Cursor can generate scaffold and report artifacts in one session; GitHub push and one
↪ screenshot remain manual. Cross-checking architecture with a second LLM
↪ (ChatGPT/Gemini) is optional but useful before Session B coding.